

React Interview Prep

useCallback & useMemo — Complete Compact Guide

Memory · Execution · Deps · When to Use · Q&A;

■ Covers

useCallback · useMemo · React.memo · Deps array · When NOT to use

■ Series

React Interview Prep — useCallback & useMemo Deep Dive

by Akshay Chavhan • React Interview Series

1. Why Do These Hooks Exist?

Every re-render, React runs the component function top to bottom — recreating every variable, object, array and function from scratch.

```
function App() {  
  const user = { name: 'Akshay' }; // new object every render  
  const nums = [1, 2, 3]; // new array every render  
  const onClick = () => {}; // new function every render  
}
```

■ **Problem: JS compares objects/arrays/functions by REFERENCE not value. `{}` `===` `{}` is false. New ref every render = child always sees 'changed' prop = unnecessary re-render.**

Reference Equality — The Root Cause

```
1 === 1 // true ■ primitives compared by VALUE  
'hi' === 'hi' // true ■  
{ } === { } // false ■ objects compared by REFERENCE  
[] === [] // false ■  
() => { } === () => { } // false ■  
  
const a = { name: 'Akshay' };  
const b = { name: 'Akshay' };  
a === b // false ■ – same value, different memory location  
const c = a; a === c // true ■ – same reference
```

2. useMemo — Memoize a VALUE

useMemo runs a function, caches the RESULT, and returns it. React only re-runs the function when dependencies change.

```
const result = useMemo(() => {  
  return expensiveComputation(a, b); // runs, result cached  
}, [a, b]); // re-run only when a or b changes
```

Use Case 1 — Heavy Computation

```
// Expensive – avoid recalc  
const sorted = useMemo(() => {  
  return data.sort(  
    (a,b) => b.score - a.score  
  );  
}, [data]);
```

Use Case 2 — Stable Reference

```
// Lightweight but new ref each  
render  
  
const user = useMemo(() => ({  
  name: 'Akshay',  
  age: 25  
}), []);  
  
// same object ref ■
```

■ **useMemo is NOT only for heavy functions. Use it anytime an object/array is passed as prop to avoid new reference every render.**

When Does useMemo Execute?

```
MOUNT → runs immediately, result cached ■  
RE-RENDER (dep unchanged) → skips, returns cache ■ ■  
RE-RENDER (dep changed) → re-runs, new cache ■
```

BUTTON CLICK (dep unchanged) → does NOT re-run ■

■ **REACT controls when useMemo runs (on render, when deps change). YOU cannot call useMemo manually. It returns a VALUE, not a function.**

3. useCallback — Memoize a FUNCTION

useCallback stores the function itself and returns the SAME reference across renders unless dependencies change. YOU call it when needed.

```
const handleDelete = useCallback((id) => {  
  setUsers(prev => prev.filter(u => u.id !== id));  
}, []); // same function reference across renders ■
```

When Does useCallback Execute?

MOUNT → function stored in memory ■ (NOT called)

RE-RENDER → same ref returned (deps unchanged)

BUTTON CLICK → function body EXECUTES ■ (YOU control this)

Click 100x → executes 100 times ■ (useCallback never blocks calls)

■ **YOU control when useCallback runs (by calling it). Deps only control memory recreation. useCallback NEVER stops the function from executing when called.**

useCallback Alone is NOT Enough — Needs React.memo

■ Wrong

```
// Child not memoized  
// re-renders anyway!  
const Child = ({ onClick }) => (  
  
  );  
// useCallback = wasted ■
```

■ Correct

```
// Child memoized ■  
const Child = React.memo(  
  ({ onClick }) => (  
  
  )  
  );  
// useCallback = works ■
```

■ **useCallback + React.memo = the pair. Without React.memo, useCallback does nothing useful.**

4. [] vs [dep] — What Changes

Deps	useCallback recreates?	useMemo re-runs?	When?
[] empty	Never after mount	Never after mount	Only on first mount
[dep]	When dep changes	When dep changes	Mount + on dep change
No array	Every render	Every render	Avoid this — pointless

Why Add a Dep? — Stale Closure Bug

■ Wrong

```
// count not in deps
// STALE – always 0!
const fn = useCallback(() => {
  console.log(count); // 0 forever
}, []); // count missing ■
```

■ Correct

```
// count in deps
// FRESH – always latest
const fn = useCallback(() => {
  console.log(count); // latest ■
}, [count]); // ■
```

■ **Rule:** If your function **READS** a variable from outside → add it to deps. State setters (setUsers, setCount) are always stable — never need to be in deps.

5. useCallback vs useMemo — Side by Side

	useCallback	useMemo
Returns	A function	A value (number, obj, array)
Who triggers it	YOU (by calling it)	REACT (on render)
Same dep + click	Executes every click ■	Does NOT re-run ■
Same dep + render	Not recreated ■	Not re-run ■
Can call manually	Yes ■	No ■ — returns value
Use for	Stable function reference	Expensive calc + stable objects
Needs React.memo?	Yes — to prevent re-render	Yes — to prevent re-render

```
// They are the same thing – different convenience API:
useCallback(fn, deps) === useMemo(() => fn, deps)
```

6. When to Use — Decision Guide

Situation	Use?	Which?
Passed to React.memo child	■ Yes	useCallback
In useEffect deps array	■ Yes	useCallback
Expensive computation on render	■ Yes	useMemo

Object/array passed to child	■ Yes	useMemo
Simple inline button handler	■ No	Neither
Child has no React.memo	■ No	Neither
Just to be safe / everywhere	■ No	Neither — adds cost

■ ***Both hooks have memory + comparison cost. Only use when you can clearly see or measure the re-render problem. Premature optimization = complexity for nothing.***

7. Interview Q&A;

Q

Beginner

Q1. What is useCallback?

→ useCallback returns a memoized function — same reference across renders unless deps change. It prevents unnecessary child re-renders when functions are passed as props to React.memo components.

Q

Beginner

Q2. What is useMemo?

→ useMemo runs a function, caches the result, and returns it. React only re-runs it when dependencies change. Used for expensive computations and to stabilize object/array references passed to children.

Q

Intermediate

Q3. Does useCallback prevent the function from executing?

→ No — useCallback only controls memory recreation. The function still executes every time you call it. Clicking a button 100 times with same deps will execute the function 100 times. Deps only decide whether a new function is created in memory on re-render.

Q

Intermediate

Q4. What is the difference between [] and [dep] in useCallback?

→ [] means the function is never recreated after mount — same reference forever. [dep] means the function is recreated whenever dep changes. Use [dep] when the function reads that variable inside it, otherwise you get a stale closure bug where the function always sees the initial value.

Q

Intermediate

Q5. Is useMemo only for heavy/expensive functions?

→ No. useMemo has two use cases: expensive computations to avoid recalculating, AND stabilizing object/array references so child components don't re-render due to new reference every render. The second use case is equally important even for lightweight values.

Q

Intermediate

Q6. Why can't you use useMemo on handleDelete?

→ useMemo memoizes a VALUE — number, object, or array. handleDelete is a function. useCallback is the correct hook for functions. Technically useMemo could wrap it, but useCallback is the clean API designed exactly for this. useCallback(fn, deps) === useMemo(() => fn, deps).

Q

Expert

Q7. Does useCallback alone prevent child re-renders?

→ No. useCallback must be paired with React.memo on the child component. useCallback ensures the function reference stays stable. React.memo ensures the child skips re-rendering when props haven't changed. Without React.memo, a stable function reference has zero effect on child re-renders.

Q**Expert****Q8. When should you NOT use useCallback or useMemo?**

→ When the child isn't wrapped in `React.memo`, when the computation is trivial, when the function is only used inline in the same component, or when you're adding it 'just to be safe'. Both hooks have memory and comparison overhead — using them blindly can make performance worse, not better.